Comparative Analysis of EfficientNetB0, MobileNetV2 and ResNet50 for Maize Leaf Disease Detection

S. Supraja
Department of Artificial Intelligence and Data Analytics
Sri Ramachandra Institute of Higher Education and Technology, Porur, Chennai, India
e0324038@sriher.edu.in

Abstract—Foliar diseases of maize are hard to tell apart in field photographs, and a classifier that returns only a label gives its user no way to judge whether to trust it. This paper describes MaizeGuard-Net Pro, a screening pipeline that wraps an ImageNet-pretrained CNN classifier with test-time augmentation, Grad-CAM attribution, entropy-based uncertainty and image-quality checks, a heatmap-derived severity indicator, and a rule-based integrated pest management (IPM) module. MobileNetV2, EfficientNetB0 and ResNet50 were fine-tuned on a four-class collection of 4,188 maize leaf images and compared on a single stratified 629-image holdout. ResNet50 scored highest (96.34 % accuracy, 95.40 % macro-F1), but its saved checkpoint is 9.45 times larger than that of MobileNetV2 (230.5 MB versus 24.4 MB), which reached 94.12 % accuracy and 92.61 % macro-F1. Because the 95 % Wilson intervals of the two models overlap, MobileNetV2 is selected for deployment under a stated rule. Between 78 % and 79 % of every model's errors were confusions between northern corn leaf blight and gray leaf spot, and MobileNetV2 and ResNet50 never labelled a diseased leaf healthy. A repeatability test also exposed a defect with wider relevance: when Grad-CAM is run with training-time augmentation layers still active, the lesion-area indicator for one image varied by 32–47 percentage points between runs. Results come from one public dataset and one split; field validity is not claimed.

Index Terms—maize leaf disease, transfer learning, MobileNetV2, Grad-CAM, predictive uncertainty, test-time augmentation, integrated pest management, model selection.

I. INTRODUCTION

Three leaf diseases account for most of the diagnostic arguments in a maize field. Northern corn leaf blight (NCLB) leaves long, cigar-shaped lesions, grey-green at first and tan later. Common rust is easier to recognise: small, raised, rusty-brown pustules scattered over both sides of the blade. Gray leaf spot (GLS) is the awkward one. Its lesions are thin tan-to-grey rectangles that stop at the veins, and young ones are easily mistaken for NCLB. Add nutrient stress, herbicide burn and insect feeding, and even an experienced scout hesitates. A smallholder with a phone and no agronomist nearby has little to fall back on.

The cost of a mistake is lopsided. Call a sick leaf healthy and treatment is delayed. Raise a false alarm and somebody sprays, or walks the field, for nothing.

Published accuracy is high. It is also easy to over-read. The PlantVillage study [1] scored above 99 % on held-out images from its own collection, then roughly 31 % on photographs from other sources. Barbedo [3] traced such gaps to small and uniform training sets, cluttered backgrounds, symptoms that change with plant age and variety, and labels that are not always right. A leaf classifier therefore needs two extras: some sign of how far to trust each answer, and a price tag in megabytes and training hours.

This paper describes MaizeGuard-Net Pro, a screening pipeline built on that view. At its centre sits a transfer-learned CNN. Around it we placed test-time augmentation, Grad-CAM attribution, an entropy-based uncertainty check, an image-quality screen, a heatmap-derived severity indicator and a rule-based integrated pest management (IPM) module. When something looks wrong, the output says so and recommends a human look. We ask three questions. How do a compact, a mid-size and a large backbone compare when trained and tested identically, counting storage and training cost as well as accuracy? Can a deployment model be chosen by a stated rule instead of the highest point estimate? And do the added modules behave well enough to show to a user, or do they fail in ways the user should be warned about?

The contributions are: (i) a controlled comparison of MobileNetV2, EfficientNetB0 and ResNet50 on one dataset, split and protocol, with Wilson intervals and an analysis of the error structure; (ii) an explicit selection rule that picks MobileNetV2 and states what is given up; (iii) a documented end-to-end pipeline, with architecture and flow diagrams and every threshold tabulated; (iv) a repeatability test showing that Grad-CAM severity is non-deterministic when Keras augmentation layers stay active, with the fix; and (v) a plain statement of what the evidence does not support.

Section II reviews related work and Section III the data. Section IV describes the method and Section V the protocol. Sections VI and VII report results and model selection, Section VIII examines the explanation and uncertainty modules, and Sections IX and X give limitations and conclusions.

II. RELATED WORK

A. Deep Learning for Plant Disease Recognition

Early plant-disease vision relied on hand-built colour and texture features. Deep learning replaced them. Mohanty et al. [1] fine-tuned networks on PlantVillage, and Ferentinos [2] ran several architectures over a larger multi-crop set. Both reported above 99 % on held-out data from the same source. Too et al. [4] compared fine-tuned architectures on PlantVillage and again got uniformly high scores. That is the real difficulty: when everything scores 99 %, small differences tell us little.

The images matter too. PlantVillage leaves were photographed on plain backgrounds [5]. PlantDoc collected web photographs with natural clutter and is much harder [6]. Genuine field imagery of maize is rarer still; Wiesner-Hanks et al. [7] annotated field photographs of northern leaf blight. The dataset we use [8] is a four-class mix of the first two sources.

B. Compact Backbones and Transfer Learning

Transfer from ImageNet is the default for small datasets. The usual recipe is to train a new head first, then unfreeze later layers while the early, general ones stay fixed [9]. We compare three backbones whose sizes differ by an order of magnitude. MobileNetV2 saves computation with depthwise separable convolutions and inverted residual blocks [10]. EfficientNet scales depth, width and resolution together [11]. ResNet50 adds

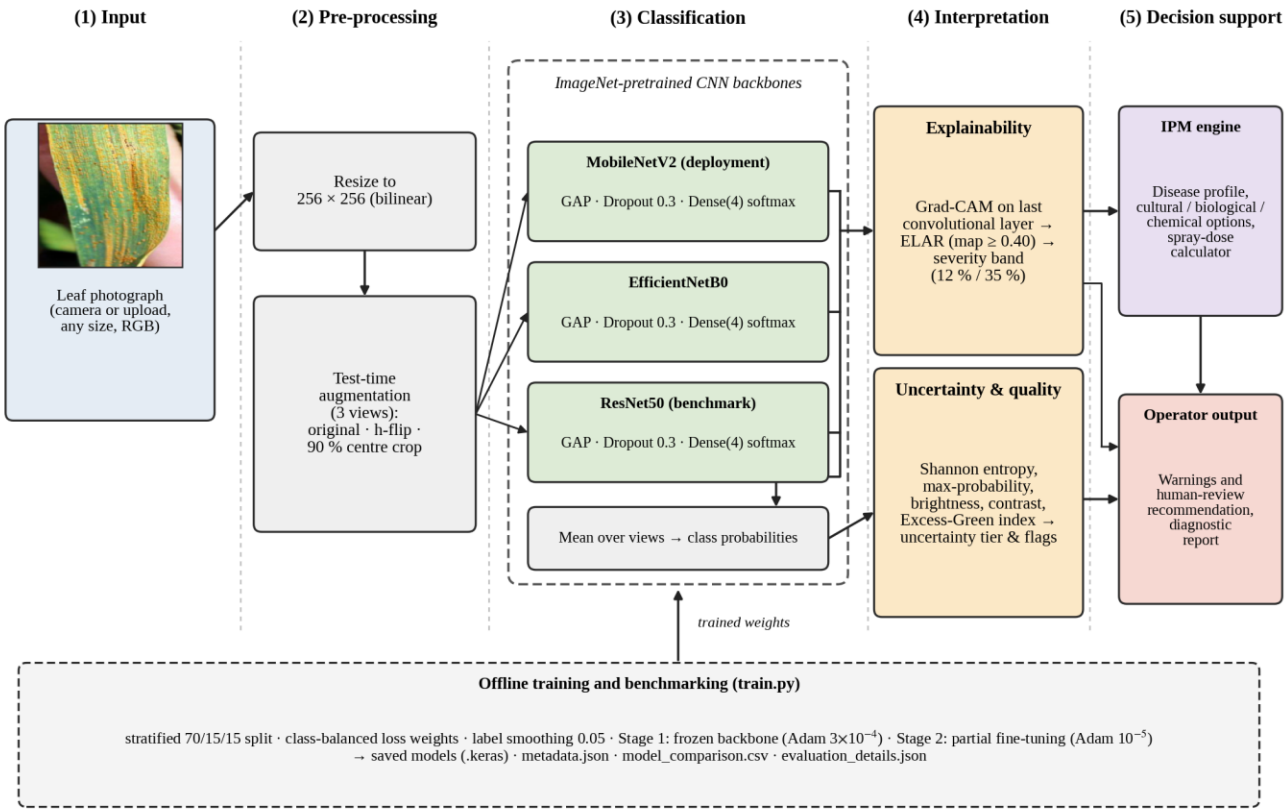


Fig. 1. Architecture of MaizeGuard-Net Pro. A photograph passes through pre-processing and test-time augmentation to the classifier; the interpretation layer adds Grad-CAM/ELAR and uncertainty/quality checks; the decision-support layer attaches IPM guidance and a human-review recommendation. The lower strip shows the offline training that produces the saved models.

identity shortcuts so that deep stacks can be trained [12]. The practical question is simple: does the larger network earn its size on maize leaves?

C. Explainability and Uncertainty

Grad-CAM [13] averages the gradient of a class score per channel over a late convolutional layer and uses the result to weight the feature maps. The output is a coarse heatmap for one class, and the network needs no modification. The heatmap says nothing about lesion size. That matters here, because we turn it into a severity number.

Softmax outputs are another trap. Modern networks are over-confident [14], so 0.9 does not mean “right nine times in ten”. Shannon entropy [15] measures how spread out a prediction is, cheaply but without calibration. Label smoothing [16] squeezes the output range further, so entropy thresholds belong to the model that produced them.

III. DATASET AND PREPROCESSING

We use the Corn or Maize Leaf Disease dataset [8], a Kaggle collection curated from PlantVillage [5] and PlantDoc [6] with unusable images removed. It holds 4,188 RGB photographs in four folders: NCLB (1,146 images), common rust (1,306), GLS (574) and healthy leaves (1,162). GLS is the minority class at 13.7 %, 2.3 times smaller than common rust. For that reason we report macro-averaged metrics beside accuracy and weight the training loss by inverse class frequency.

The split was made once, with a stratified splitter and seed 42: 15 % for testing, then 15 % of the total for validation. That leaves 2,930 images for training, 629 for validation and 629 for testing (Table I). Stratification matters most for GLS. The test partition holds only 86 GLS leaves, so a single error moves GLS recall by 1.2 points. The test set was touched only for the final comparison; validation data drove early stopping.

TABLE I
COMPOSITION OF THE DATASET AND TEST PARTITION

Class	Images	Share	Test images
Northern corn leaf blight	1,146	27.4 %	172
Common rust	1,306	31.2 %	196
Gray leaf spot	574	13.7 %	86
Healthy	1,162	27.7 %	175
Total	4,188	100 %	629

Each image is decoded to three-channel RGB and resized straight to 256 × 256 pixels with bilinear interpolation. There is no cropping or padding. Non-square photographs are therefore slightly stretched, which we accepted so that lesions at the leaf edge are never cut away. Backbone-specific intensity normalisation is built into each saved model. Random flips, rotations, zooms and contrast changes are applied during training only.

One caveat. The collection mixes laboratory-style and web-sourced images, so image style may correlate with class. A

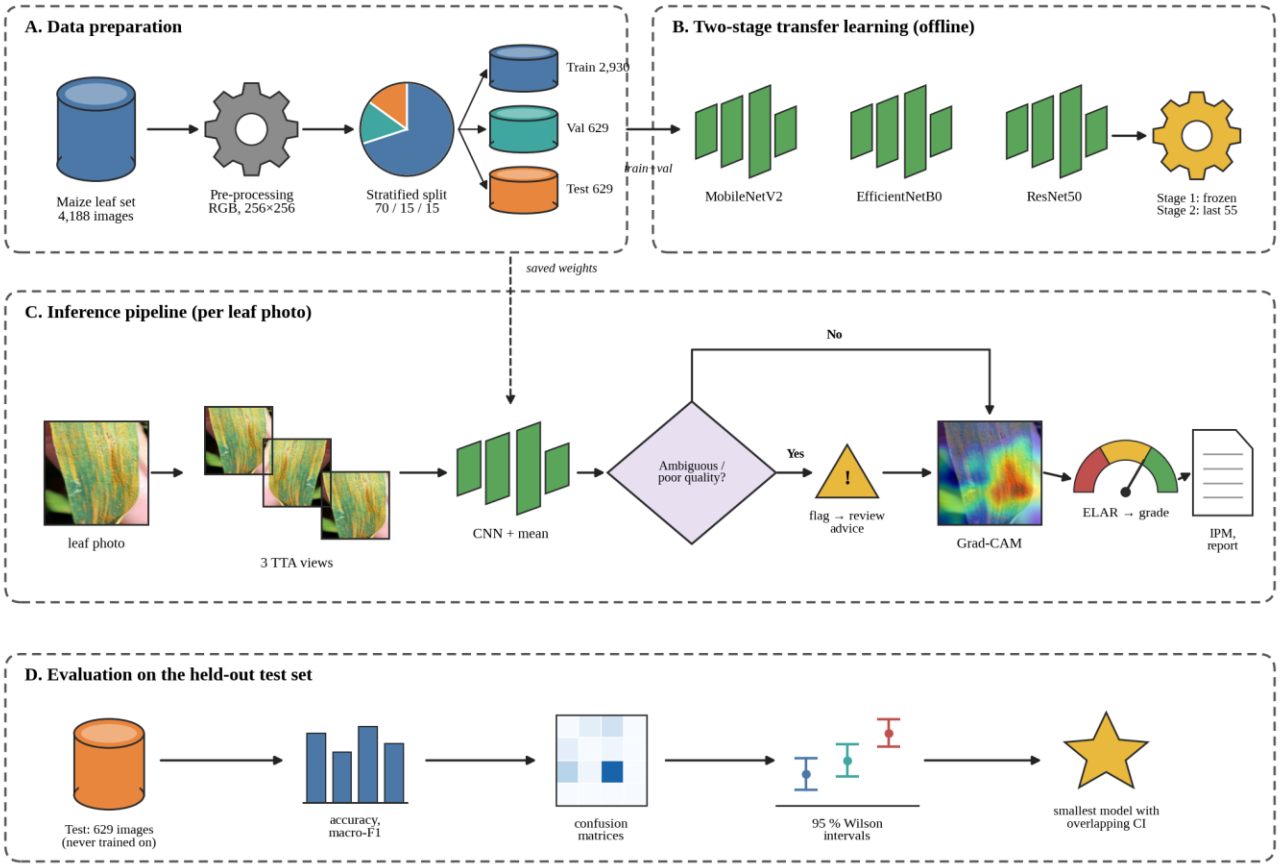


Fig. 2. Workflow of MaizeGuard-Net Pro. A: data preparation and stratified split. B: two-stage transfer learning of three backbones. C: inference on one photograph; a quality or ambiguity flag adds a warning but does not stop the pipeline. D: evaluation on the held-out test set and model selection.

random split cannot expose such shortcuts [3]. The scores below describe this dataset, not farms.

IV. SYSTEM ARCHITECTURE AND METHOD

A. Pipeline Overview

Fig. 1 shows the architecture and Fig. 2 the full workflow, from data preparation to evaluation. A diagnosis runs as follows. The photograph is resized and expanded into three views. The classifier averages their class probabilities. A quality and uncertainty module inspects both the pixels and the probability vector. Grad-CAM then localises evidence and converts the map into a lesion-area indicator and severity band. Finally the IPM module attaches management text and a spray-volume calculation.

Warnings do not stop the pipeline. A flagged image still receives a prediction, a heatmap and advice, plus a message recommending human review. The classifier may be any one of the three trained backbones or the optional ensemble of Section IV-C.

B. Classifier and Transfer Learning

Each classifier (Fig. 3) is an ImageNet-pretrained backbone without its original top, followed by global average pooling, dropout at 0.3 and a four-unit softmax layer. Training has two stages. In Stage 1 the backbone is frozen and only the head is trained, with Adam [17] at 3×10^{-4}. The head goes first so that its random weights cannot push large gradients into the pretrained filters. In Stage 2 the last 55 backbone layers are unfrozen and training continues at 10^{-5}. Batch-normalisation layers stay frozen throughout, because with a batch size of 32 their statistics would be noisy.

Loss is categorical cross-entropy with class weights inversely proportional to class frequency, which follows directly from GLS having 574 images against 1,306 for rust. Label smoothing is kept small at 0.05 [16]. A heavier setting would compress the output range on which the entropy rules of Section IV-E depend. Early stopping on validation accuracy (patience 3, best weights restored) and a learning-rate reduction on plateau control the schedule. Table II lists the shared configuration.

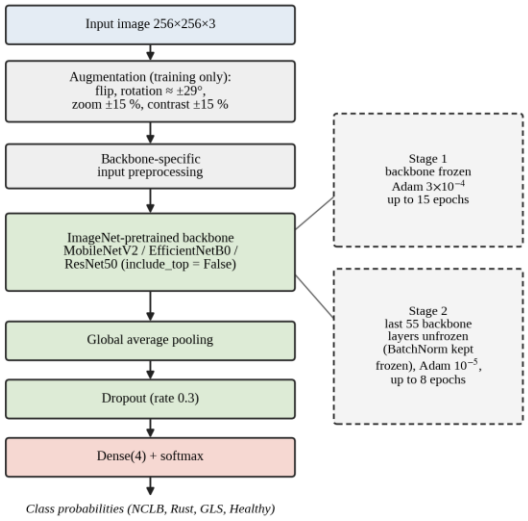


Fig. 3. Classifier architecture and two-stage transfer-learning schedule shared by MobileNetV2, EfficientNetB0 and ResNet50.

TABLE II
TRAINING CONFIGURATION COMMON TO ALL THREE BACKBONES

Setting	Value
Input	256 × 256 × 3, RGB
Backbones	MobileNetV2, EfficientNetB0, ResNet50 (ImageNet weights)
Head	Global average pooling, dropout 0.3, Dense(4) softmax
Loss	Cross-entropy, label smoothing 0.05, balanced class weights
Optimiser	Adam; 3×10^{-4} (Stage 1), 10^{-5} (Stage 2)
Epoch limits	15 (Stage 1), 8 (Stage 2); batch size 32
Fine-tuned layers	Last 55 backbone layers, BatchNorm kept frozen
Callbacks	Early stopping (patience 3); learning rate × 0.5 on plateau (patience 2)
Augmentation	Flip, rotation ≈ ±29°, zoom ±15 %, contrast ±15 % (training only)
Split, seed	Stratified 2,930 / 629 / 629; seed 42

C. Test-Time Augmentation and Aggregation

At inference the photograph is evaluated in three views: the original, its horizontal mirror, and a centre crop that removes 5 % from each border. The trim is deliberately mild, so lesions near the leaf edge survive it. The class probabilities of the views are averaged (1).

$$\bar{\mathbf{p}} = \frac{1}{V}\sum_{v=1}^{V} f_\theta(\mathbf{x}^{(v)}), \quad V = 3 \tag{1}$$

The pipeline also offers a weighted soft vote over the three backbones (2). The weights are fixed in the source code: 0.371 for MobileNetV2, 0.367 for ResNet50 and 0.262 for EfficientNetB0. They do not follow the macro-F1 ordering of Section VI, where MobileNetV2 ranks lowest, and the ensemble was never scored on the test set. We therefore claim no performance for it.

$$\bar{\mathbf{p}}_{\text{ens}} = \frac{\sum_m w_m \bar{\mathbf{p}}_m}{\sum_m w_m} \tag{2}$$

D. Grad-CAM, ELAR and Severity

For a target class c (in the current implementation the top-ranked class of the un-augmented view), Grad-CAM computes channel weights α_k^c by averaging the gradient of the class score y^c over the spatial positions of the last convolutional feature map A^k. The map L^c is the rectified weighted sum (3) [13]. For a 256 × 256 input it is only 8 × 8. We min–max normalise it to give $\hat{L}^c$, upsample it bicubically to photograph size and blend it with the image. The Estimated Lesion Area Ratio (ELAR) is the percentage of pixels in the normalised map that reach 0.40 (4).

$$\alpha_k^c = \frac{1}{Z}\sum_i\sum_j \frac{\partial y^c}{\partial A_{ij}^k}, \qquad L^c = \mathrm{ReLU}\left(\sum_k \alpha_k^c A^k\right) \tag{3}$$

$$\mathrm{ELAR} = \frac{100}{HW}\sum_{u=1}^{H}\sum_{v=1}^{W} \mathbf{1}\left[\hat{L}^c(u,v) \geq 0.40\right] \tag{4}$$

A disease prediction is graded Mild below 12 % ELAR, Moderate from 12 % to below 35 %, and Severe from 35 % upward. A Healthy prediction gets grade 0 whatever the ELAR. The two cut-offs and the 0.40 threshold are design choices; none was fitted to lesion annotations. The first version of this routine also hid a bug, described in Section VIII-B.

E. Uncertainty and Quality Screening

Two kinds of check run on every image (Table III). The first concerns pixels: very low contrast, extreme darkness or brightness, and a strongly negative mean excess-green index (6) [18] together with a weak green channel, which suggests a non-vegetation image. The second concerns the probability vector: normalised Shannon entropy (5) and the largest class probability. A triggered rule raises a flag and a human-review recommendation. Separately, a three-level tier (high certainty, moderate certainty, high uncertainty) is shown to the user. Flag and tier use different thresholds and can disagree, as Section VIII-A shows. All thresholds were set by hand. None was calibrated on held-out data.

$$H(\mathbf{p}) = -\sum_{c=1}^{C} p_c \log_2 p_c, \qquad H_n = \frac{H(\mathbf{p})}{\log_2 C} \tag{5}$$

$$\mathrm{ExG} = 2G - R - B \tag{6}$$

TABLE III
DECISION RULES IMPLEMENTED IN THE PIPELINE

Check	Condition	Outcome
Contrast	Pixel standard deviation < 15	Flag
Exposure	Mean intensity < 25 or > 240	Flag
Vegetation	Mean ExG < −40 and mean G < 40	Flag
Ambiguity	$H_n > 0.65$ or $p_{max} < 0.50$	Flag
Tier: high certainty	$H_n < 0.25$ and $p_{max} \geq 0.85$	Green
Tier: moderate	$H_n < 0.55$ and $p_{max} \geq 0.60$	Amber
Tier: high uncertainty	Otherwise	Red
Severity	ELAR < 12 % / < 35 % / ≥ 35 %	Mild / Mod. / Severe

F. IPM Module and Dose Calculator

The IPM module is a static knowledge base keyed by the predicted class. For each disease it stores the causal organism, a symptom description, the weather that favours development, cultural and biological control options, and two chemical options with an active-ingredient class, an application rate per acre, a pre-harvest interval and a re-entry interval. The healthy class maps to scouting and nutrition advice.

A calculator converts the rate to field scale. For a field of A acres sprayed at v_w litres per acre from tanks of capacity T litres,

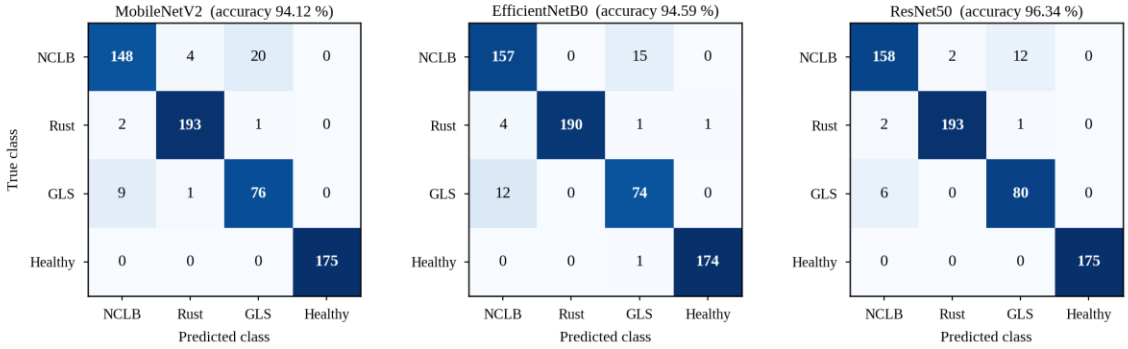


Fig. 4. Confusion matrices on the 629-image test partition (rows: true class; columns: predicted class). NCLB: northern corn leaf blight; GLS: gray leaf spot.

the tank loads and the product per tank follow from (7), where q is the product rate per acre. A 2.5-acre field sprayed at 200 L per acre from a 16 L knapsack needs 31.25 tank loads.

$$n_{\text{tank}} = \frac{A\, v_w}{T}, \qquad d_{\text{tank}} = \frac{A\, q}{n_{\text{tank}}} \tag{7}$$

The entries are text written into the source code. Product names, rates and intervals depend on local registration and must be checked against the label before any use. The module is advisory, not a prescription.

V. EXPERIMENTAL PROTOCOL

The three backbones were trained from the same split, seed and configuration (Table II) and evaluated once on the 629-image test partition. We report accuracy and macro-averaged precision, recall and F1. Macro averages weight the four classes equally, so they expose weakness on GLS. Each accuracy carries a 95 % Wilson score interval (8) [19], with $\hat{p}$ the observed accuracy, $n = 629$ and $z = 1.96$.

$$\text{CI}_{95\%} = \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \quad z = 1.96 \tag{8}$$

Overlap between two such intervals is a conservative, informal sign that 629 images cannot resolve the difference. A paired test such as McNemar's [20] would be more sensitive, but it needs per-image predictions, and those were not saved. Cost is reported as saved model size and the wall-clock training time of both stages. Models were built with TensorFlow and Keras [21]. Each configuration was trained once, so there is no variance across seeds to report.

VI. CLASSIFICATION RESULTS

A. Overall Performance

Table IV and Fig. 5(a) give the holdout results. ResNet50 leads on every metric: 96.34 % accuracy (606 of 629 images correct) and 95.40 % macro-F1. EfficientNetB0 follows with 94.59 % and 93.13 %, then MobileNetV2 with 94.12 % and 92.61 %. Fourteen images separate ResNet50 from MobileNetV2, a gap of 2.22 points. The Wilson intervals overlap for all three pairs. With 629 test images the ordering is suggestive, not established.

TABLE IV
HOLDOUT PERFORMANCE ON 629 TEST IMAGES (%)

Model	Acc.	95 % CI	Macro-P	Macro-R	Macro-F1
MobileNetV2	94.12	92.0–95.7	92.23	93.22	92.61
EfficientNetB0	94.59	92.5–96.1	92.87	93.42	93.13
ResNet50	96.34	94.6–97.6	95.04	95.84	95.40

B. Error Structure

Fig. 4 shows the confusion matrices. Healthy leaves are almost never mistaken for disease, and diseased leaves almost never for healthy. MobileNetV2 and ResNet50 make no disease-as-healthy error among the 454 diseased test images; EfficientNetB0 makes one (Table V). The errors pile up on a single pair. Confusion between NCLB and GLS accounts for 78–79 % of every model's errors. Both diseases give elongated tan to grey lesions, a plausible visual reason, though this study has no lesion-level analysis to confirm it.

GLS is therefore the weakest class (Table VI). Its F1 is 83.06 % for MobileNetV2 and 89.39 % for ResNet50. MobileNetV2 reaches 88.37 % recall on GLS but only 78.35 % precision, because it over-calls GLS on NCLB leaves: 20 of the 172 NCLB test images.

TABLE V
ERROR STRUCTURE ON THE TEST PARTITION

Model	Errors	NCLB↔GLS	Disease → healthy	Healthy → disease
MobileNetV2	37	29 (78.4 %)	0	0
EfficientNetB0	34	27 (79.4 %)	1	1
ResNet50	23	18 (78.3 %)	0	0

TABLE VI
PER-CLASS F1 ON THE TEST PARTITION (%)

Class	MobileNetV2	EffNetB0	ResNet50
NCLB	89.43	91.01	93.49
Common rust	97.97	98.45	98.72
Gray leaf spot	83.06	83.62	89.39
Healthy	100.00	99.43	100.00

VII. MODEL SELECTION AND DEPLOYMENT TRADE-OFFS

Accuracy, size and training time pull in different directions (Fig. 5(b), Table VII). ResNet50 is the most accurate and by far

the largest: 23.6 M parameters and a 230.5 MB file. MobileNetV2 has 2.26 M parameters and a 24.4 MB file, 9.45 times smaller. EfficientNetB0 sits between them, and it trained fastest. No model wins on every objective, so a selection rule is needed.

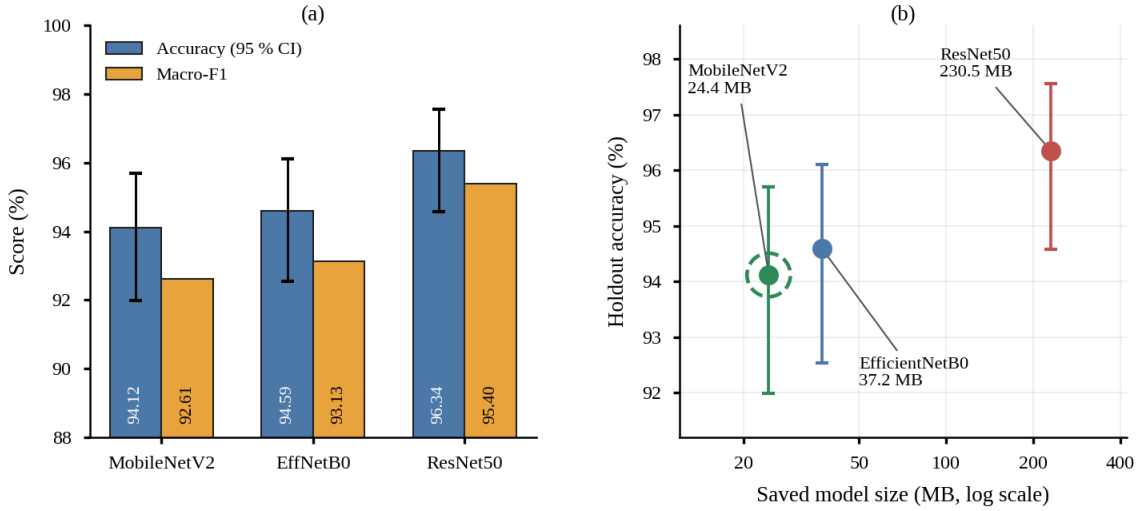


Fig. 5. Holdout results. (a) Accuracy with 95 % Wilson intervals and macro-F1. (b) Accuracy against saved model size (log scale); the dashed ring marks the model selected for deployment.

Ours is this: among the models whose accuracy interval overlaps that of the most accurate model, take the smallest. On the present evidence the rule selects MobileNetV2. Its interval (92.0–95.7 %) overlaps ResNet50's (94.6–97.6 %), and its file is the smallest. This is a judgement about what the data can resolve. It is not a claim that MobileNetV2 is as accurate as ResNet50. A 2.22-point difference may well be real, and where it matters, ResNet50 remains the better choice.

Two caveats apply to the cost columns. Training time (24,361 s for ResNet50, 7,300 s for MobileNetV2, 3,561 s for EfficientNetB0) does not follow model size. The epochs run before early stopping and the load on the training machine were not recorded, so it is not a clean cost measure. File sizes are those of saved training checkpoints, 2.3 to 2.7 times larger than the float32 weights alone (Table VII), which is consistent with stored training state. Deployable size should be re-measured after export to a mobile format.

No latency, memory or energy was measured on a target device. The case for MobileNetV2 on a handheld therefore rests on parameter count and file size alone.

TABLE VII
DEPLOYMENT-RELEVANT COMPARISON

Criterion	MobileNetV2	EffNetB0	ResNet50
Parameters (M)	2.26	4.05	23.60
Float32 weights (MB)	9.0	16.2	94.4
Saved checkpoint (MB)	24.39	37.21	230.53
Accuracy (%)	94.12	94.59	96.34
Macro-F1 (%)	92.61	93.13	95.40
Disease → healthy errors	0	1	0
Training time (s)	7,300	3,561	24,361
Latency, memory, energy	not measured on a target device		

VIII. BEHAVIOUR OF THE EXPLANATION AND UNCERTAINTY MODULES

A. Qualitative Case Study

This dataset offers no ground truth for heatmaps, severity or uncertainty. We therefore examined the modules qualitatively on the four photographs shipped with the project, one per class, using MobileNetV2 with test-time augmentation (Fig. 6). We did not confirm that these images lie outside the training data. They show behaviour, not accuracy. All four are classified correctly. Four observations follow.

(1) The NCLB pair (a) is predicted confidently (0.94, H_n = 0.18). Yet the heatmap peaks on the lower, still partly green leaf and gives little weight to most of the necrotic upper one. The map shows where the network found evidence. It does not show how much tissue is damaged.

(2) The rust leaf (b) is a near tie. For the un-augmented view the top class is GLS (0.47), with rust a close second (0.45). Averaging the three views flips the decision to rust (0.61). With $H_n = 0.62$ the tier reads "high uncertainty", yet no flag is raised, since neither flag threshold is crossed. Worse, the heatmap is computed for the un-augmented view's top class, GLS. The explanation shown belongs to a class other than the one reported.

(3) The GLS leaf (c) is reported at 0.48 and is flagged, as intended.

(4) The healthy leaf (d) yields an ELAR of 65.0 %, higher than any diseased image. The value is discarded because a Healthy prediction is graded 0. It still shows that ELAR measures how widely the network's attention spreads, not how much lesion there is. The Severe grades of (a)–(c) should be read in that light.

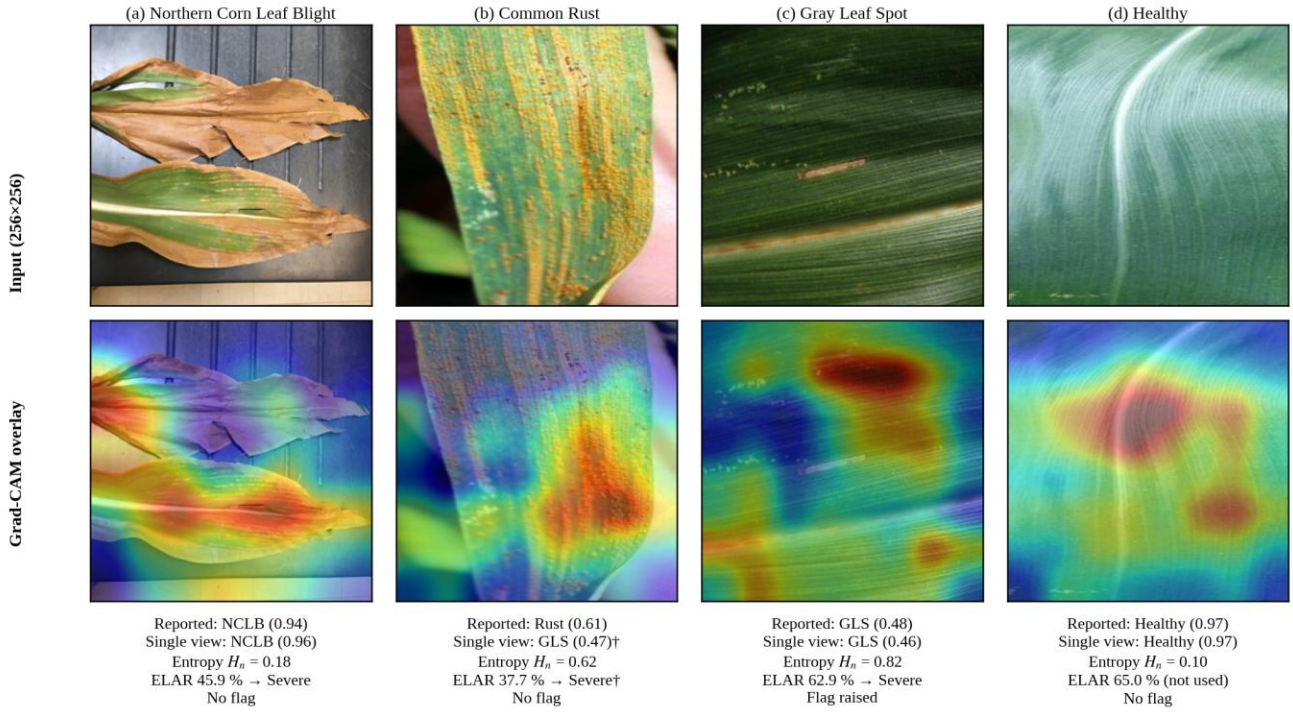


Fig. 6. Pipeline output for one sample image per class (MobileNetV2 with test-time augmentation): input, Grad-CAM overlay, reported class and probability, class of the un-augmented view, normalised entropy Hn, ELAR with severity grade, and review flag. † The heatmap targets the top class of the un-augmented view (GLS), not the reported class (rust).

B. Repeatability of the Severity Indicator

Running the Grad-CAM routine twice on the same leaf produced two different severity grades. That was the first sign of trouble, so we tested it properly.

In the original code the manual forward pass through the layers before the backbone calls them without a *training* argument. In our environment (TensorFlow 2.21 with Keras 3) the random augmentation layers in that stack then stay active. Each call sees a randomly flipped, rotated, zoomed and contrast-shifted copy of the image. We called the routine 20 times per image. Table VIII and Fig. 7 show the result: standard deviations of 9–10 percentage points and ranges of 32–47 points. That is enough to move one image across severity cut-offs. The NCLB leaf scored Moderate at 15.6 % and Severe at 51.5 %. The rust leaf scored Mild at 9.2 % and Severe at 41.5 %.

The standard prediction call is unaffected, because it runs in inference mode. Passing *training* = False to every layer call removes the randomness. Ten repeated runs per image then returned identical values, and those are the values in Fig. 6. Any system that reuses Keras models containing augmentation layers inside a hand-written gradient routine should check for this.

TABLE VIII
ELAR REPEATABILITY OVER 20 RUNS (%)

Image	Min	Max	Mean ± SD	Fixed (deterministic)
NCLB	15.6	51.5	31.1 ± 9.9	45.9
Common rust	9.2	41.5	22.9 ± 9.0	37.7
Gray leaf spot	18.2	64.7	32.7 ± 10.2	62.9
Healthy	31.5	74.0	62.1 ± 9.2	65.0

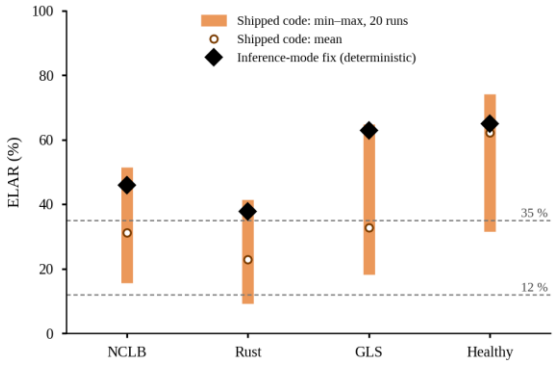


Fig. 7. ELAR for the four sample images. Bars show the min–max range over 20 runs of the original code; diamonds show the deterministic value with augmentation layers in inference mode. Dashed lines mark the 12 % and 35 % severity cut-offs.

IX. LIMITATIONS

Data and statistics. We used one public dataset, one split and one training run per model. The dataset mixes laboratory-style and web images. No external or field validation was done, so field accuracy is unknown and probably lower [1,3]. The intervals describe sampling error on 629 images, not variation across seeds or splits. Interval overlap is not a significance test,

and the paired comparison needed to rank the models was not possible.

Ensemble. The weighted ensemble was not evaluated, and its weights are untuned.

Severity. ELAR is a heatmap statistic. It has not been compared with pixel-level lesion masks or expert scores, its thresholds are arbitrary, and the case study shows that it can contradict visible damage. It should not drive treatment decisions. The heatmap target class can also differ from the reported class when test-time augmentation changes the decision.

Uncertainty. Entropy and probability thresholds are hand-set and uncalibrated, and label smoothing changes their meaning. We ran no calibration analysis, such as expected calibration error, and no test on out-of-distribution inputs such as other crops.

IPM and deployment. The IPM content is static text. Registered products, rates and intervals differ between jurisdictions, and an agronomist must confirm any recommendation. No latency, memory or energy was measured on target hardware, saved sizes include training state, and training times were not controlled.

X. CONCLUSION

MaizeGuard-Net Pro pairs a transfer-learned maize leaf classifier with test-time augmentation, Grad-CAM, entropy and quality screening, a heatmap-based severity indicator and rule-based IPM advice. On a 629-image holdout from a public four-class dataset, ResNet50 scored highest (96.34 % accuracy). MobileNetV2 reached 94.12 % with a model 9.45 times smaller, and because the intervals overlap, it is selected under a stated rule. Most errors of all three models are NCLB–GLS confusions. MobileNetV2 and ResNet50 never called a diseased leaf healthy.

The accompanying modules call for more caution than the classifier. ELAR does not measure lesion extent, and its original implementation was not repeatable until the augmentation layers were forced into inference mode.

Future work includes validation on field photographs, saved per-image predictions for paired testing, multi-seed training, a calibration analysis, comparison of ELAR with lesion masks, evaluation of the ensemble, and latency and memory measurement of a quantised MobileNetV2 on a representative phone.

REFERENCES

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Front. Plant Sci.*, vol. 7, Art. no. 1419, Sep. 2016.

[2] K. P. Ferentinos, “Deep learning models for plant disease detection and diagnosis,” *Comput. Electron. Agric.*, vol. 145, pp. 311–318, Feb. 2018.

[3] J. G. A. Barbedo, “Factors influencing the use of deep learning for plant disease recognition,” *Biosyst. Eng.*, vol. 172, pp. 84–91, Aug. 2018.

[4] E. C. Too, L. Yujian, S. Njuki, and L. Yingchun, “A comparative study of fine-tuning deep learning models for plant disease identification,” *Comput. Electron. Agric.*, vol. 161, pp. 272–279, Jun. 2019.

[5] D. P. Hughes and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” arXiv:1511.08060, 2015.

[6] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, “PlantDoc: A dataset for visual plant disease detection,” in *Proc. 7th ACM IKDD CoDS and 25th COMAD*, Hyderabad, India, 2020, pp. 249–253.

[7] T. Wiesner-Hanks *et al.*, “Image set for deep learning: Field images of maize annotated with disease symptoms,” *BMC Res. Notes*, vol. 11, Art. no. 440, 2018.

[8] S. Ghose, “Corn or maize leaf disease dataset,” Kaggle. [Online]. Available: https://www.kaggle.com/datasets/smaranjitghose/corn-or-maize-leaf-disease-dataset

[9] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How transferable are features in deep neural networks?” in *Adv. Neural Inf. Process. Syst.*, vol. 27, 2014, pp. 3320–3328.

[10] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 4510–4520.

[11] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105–6114.

[12] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[13] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618–626.

[14] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321–1330.

[15] C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948.

[16] R. Müller, S. Kornblith, and G. E. Hinton, “When does label smoothing help?” in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 4694–4703.

[17] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[18] D. M. Woebbecke, G. E. Meyer, K. Von Bargen, and D. A. Mortensen, “Color indices for weed identification under various soil, residue, and lighting conditions,” *Trans. ASAE*, vol. 38, no. 1, pp. 259–269, 1995.

[19] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[20] T. G. Dietterich, “Approximate statistical tests for comparing supervised classification learning algorithms,” *Neural Comput.*, vol. 10, no. 7, pp. 1895–1923, 1998.

[21] M. Abadi *et al.*, “TensorFlow: Large-scale machine learning on heterogeneous systems,” arXiv:1603.04467, 2015.